14.2 入站消息处理流水线

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~280,000 tokens，输出 ~20,000 tokens（本章合计）

一条消息从外部通道（Telegram、WhatsApp 等）到达 Gateway 时，需要经过一系列处理步骤才能到达 AI Agent。这个处理流程就是**入站消息流水线**（Inbound Message Pipeline），负责消息格式统一、发送者身份识别、权限过滤和路由决策。

14.2.1 消息接收与规范化

聊天类型规范化

不同通道对消息类型的命名各不相同：Telegram 区分"private"和"group"，Discord 使用"DM"和"GUILD_TEXT"，Slack 使用"im"和"channel"。 `src/channels/chat-type.ts` 将这些不同的命名统一为三种标准类型：

```
// src/channels/chat-type.ts
export type NormalizedChatType = "direct" | "group" | "channel";

export function normalizeChatType(raw?: string): NormalizedChatType | undefined {
  const value = raw?.trim().toLowerCase();
  if (!value) return undefined;
  if (value === "direct" || value === "dm") return "direct";
  if (value === "group") return "group";
  if (value === "channel") return "channel";
  return undefined;
}
```

原始类型	规范化类型	说明
`direct / dm`	`"direct"`	一对一私聊
`group`	`"group"`	多人群组
`channel`	`"channel"`	频道（如 Discord 频道、Telegram 频道）

这种规范化使得 Gateway 的后续逻辑可以统一处理，不需要关心消息具体来自哪个平台。

14.2.2 发送者身份识别

在群组场景中，仅仅知道消息来自"哪个群"是不够的——还需要知道**谁**发了这条消息。 `src/channels/sender-identity.ts` 提供了发送者身份的校验逻辑：

```ts
// src/channels/sender-identity.ts
export function validateSenderIdentity(ctx: MsgContext): string[] {
  const issues: string[] = [];
  const chatType = normalizeChatType(ctx.ChatType);
  const isDirect = chatType === "direct";

  const senderId = ctx.SenderId?.trim() || "";
  const senderName = ctx.SenderName?.trim() || "";
  const senderUsername = ctx.SenderUsername?.trim() || "";
  const senderE164 = ctx.SenderE164?.trim() || "";

  // 群组消息必须有发送者身份
  if (!isDirect) {
    if (!senderId && !senderName && !senderUsername && !senderE164) {
      issues.push("missing sender identity (SenderId/SenderName/SenderUsername/SenderE1
    }
  }

  // E.164 格式校验（国际电话号码标准）
  if (senderE164) {
    if (!/^\+\d{3,}$/.test(senderE164)) {
      issues.push(`invalid SenderE164: ${senderE164}`);
    }
  }

  // 用户名不应包含 @ 或空白
  if (senderUsername) {
    if (senderUsername.includes("@")) {
      issues.push(`SenderUsername should not include "@": ${senderUsername}`);
    }
    if (/\s/.test(senderUsername)) {
      issues.push(`SenderUsername should not include whitespace: ${senderUsername}`);
    }
  }

  return issues;
}
```

> **衍生解释**：E.164 是国际电信联盟（ITU）制定的电话号码标准格式，格式为 `+` 加国家代码加电话号码，例如 `+8613812345678`（中国）或 `+14155551234`（美国）。OpenClaw 中 WhatsApp 和 Signal 通道使用这种格式标识用户。

OpenClaw 的消息上下文（`MsgContext`）提供了四种发送者标识字段：

字段	说明	示例
`SenderId`	平台内部用户 ID	`123456789` （Telegram）、 `U0123ABC` （Slack）
`SenderName`	用户显示名	`Alice` 、 `张三`
`SenderUsername`	用户名（不含 @）	`alice_dev` 、 `zhangsan`
`SenderE164`	E.164 格式电话号码	`+8613812345678`

私聊消息不要求发送者身份（因为只有一个对话方）；但群组消息至少需要一个身份字段，否则 Agent 无法区分不同的群成员。

14.2.3 白名单匹配

出于安全考虑，OpenClaw 默认不会响应陌生人的消息。 `allowFrom` 白名单机制控制哪些用户或群组可以与 Agent 交互。

`src/channels/allowlists/resolve-utils.ts` 提供了白名单的合并工具：

```
// src/channels/allowlists/resolve-utils.ts
export function mergeAllowlist(params: {
  existing?: Array<string | number>;
  additions: string[];
}): string[] {
  const seen = new Set<string>();
  const merged: string[] = [];
  const push = (value: string) => {
    const normalized = value.trim();
    if (!normalized) return;
    const key = normalized.toLowerCase();
    if (seen.has(key)) return;  // 去重
    seen.add(key);
    merged.push(normalized);
  };
  for (const entry of params.existing ?? []) push(String(entry));
  for (const entry of params.additions) push(entry);
  return merged;
}
```

每个通道的 Dock 层定义了自己的白名单解析和格式化逻辑。例如 WhatsApp 需要将号码规范化为 E.164 格式，Telegram 使用用户 ID，Discord 使用用户 ID 的字符串形式。

白名单支持特殊值 `"*"` ，表示允许所有用户。这对于开放的群组机器人很有用，但需要谨慎使用——因为这意味着任何人都可以消耗你的 LLM 配额。

14.2.4 @提及门控（Mention Gating）

在群组中，AI 助手通常不应响应每一条消息，而只应在被 @提及时才回复。 `src/channels/mention-gating.ts` 实现了这种门控逻辑：

```
// src/channels/mention-gating.ts
export type MentionGateParams = {
  requireMention: boolean;        // 是否要求被提及
  canDetectMention: boolean;      // 通道是否能检测提及
  wasMentioned: boolean;          // 是否实际被提及
  implicitMention?: boolean;      // 隐式提及（如回复 AI 的消息）
  shouldBypassMention?: boolean; // 是否绕过提及要求
};

export function resolveMentionGating(params: MentionGateParams): MentionGateResult {
  const implicit = params.implicitMention === true;
  const bypass = params.shouldBypassMention === true;
  const effectiveWasMentioned = params.wasMentioned || implicit || bypass;
  // 当要求提及、能检测提及、且未被提及时，跳过该消息
  const shouldSkip = params.requireMention && params.canDetectMention && !effectiveWasM
  return { effectiveWasMentioned, shouldSkip };
}
```

门控逻辑考虑了三种"被提及"的方式：

1. **显式提及**（ `wasMentioned` ）：用户在消息中 @了机器人
2. **隐式提及**（ `implicitMention` ）：用户回复了机器人的消息
3. **绕过提及**（ `shouldBypassMention` ）：授权用户发送了控制命令（如 `/new` ）

`resolveMentionGatingWithBypass()` 进一步处理了命令绕过的场景：

```ts
// src/channels/mention-gating.ts
export function resolveMentionGatingWithBypass(
  params: MentionGateWithBypassParams,
): MentionGateWithBypassResult {
  // 绕过条件：在群组中 + 要求提及 + 未被提及 + 允许文本命令 + 有控制命令 + 已授权
  const shouldBypassMention =
    params.isGroup &&
    params.requireMention &&
    !params.wasMentioned &&
    !(params.hasAnyMention ?? false) &&
    params.allowTextCommands &&
    params.commandAuthorized &&
    params.hasControlCommand;
  return {
    ...resolveMentionGating({ ...params, shouldBypassMention }),
    shouldBypassMention,
  };
}
```

也就是说，在群组中，即使没有 @机器人，授权用户发送 `/new` 或 `/reset` 等控制命令也会被处理。

14.2.5 命令门控（Command Gating）

`src/channels/command-gating.ts` 控制谁可以执行命令，与提及门控配合工作：

```ts
// src/channels/command-gating.ts
export function resolveControlCommandGate(params: {
  useAccessGroups: boolean;        // 是否使用访问组
  authorizers: CommandAuthorizer[]; // 授权器列表
  allowTextCommands: boolean;      // 是否允许文本命令
  hasControlCommand: boolean;      // 消息是否包含控制命令
}): { commandAuthorized: boolean; shouldBlock: boolean } {
  const commandAuthorized = resolveCommandAuthorizedFromAuthorizers({
    useAccessGroups: params.useAccessGroups,
    authorizers: params.authorizers,
  });
  // 如果消息包含控制命令但发送者未授权，则阻止
  const shouldBlock = params.allowTextCommands
    && params.hasControlCommand
    && !commandAuthorized;
  return { commandAuthorized, shouldBlock };
}
```

`CommandAuthorizer` 是一个简单的接口：

```ts
export type CommandAuthorizer = {
  configured: boolean;  // 是否配置了该授权器
  allowed: boolean;     // 当前用户是否被允许
};
```

多个授权器形成一个链——只要其中一个配置了且允许，就通过授权。当 `useAccessGroups` 关闭时，授权策略由 `modeWhenAccessGroupsOff` 决定：`"allow"` 表示全部放行，`"deny"` 表示全部拒绝，`"configured"` 表示只看已配置的授权器。

本节小结

入站消息处理流水线是 OpenClaw 安全和灵活性的第一道防线：

1. **类型规范化**：将不同通道的消息类型统一为 `direct` / `group` / `channel` 三种
2. **身份识别**：群组消息必须携带发送者身份，支持四种标识字段（ID / 名称 / 用户名 / E.164）
3. **白名单过滤**：默认拒绝陌生人消息，通过 `allowFrom` 配置显式授权
4. **提及门控**：群组中只响应被 @提及的消息，但授权用户的控制命令可以绕过
5. **命令门控**：控制命令（如 `/new` 、 `/reset` ）需要额外的授权检查

这套流水线确保了只有合法的消息才能到达 Agent，同时为不同场景（私聊/群组/频道）提供了差异化的处理策略。